

Programming in C

UNIX System Calls and Subroutines using C.

© A. D. Marshall 1994-2005

Substantially Updated March 1999

 NetGuide
Gold Site

Next: [Contents](#)

Download Postscript Version of Notes

[Click Here to Download](#) Course Notes. Local Students Only.

Algorithm Animations

[Direct link to Java Algorithm Animations \(C related\)](#)

C COURSEWARE

Lecture notes + integrated exercises, solutions and marking

- [Contents](#)
- [The Common Desktop Environment](#)
 - [The front panel](#)
 - [The file manager](#)
 - [The application manager](#)
 - [The session manager](#)
 - [Other CDE desktop tools](#)
 - [Application development tools](#)
 - [Application integration](#)
 - [Windows and the Window Manager](#)
 - [The Root Menu](#)
 - [Exercises](#)

- [C/C++ Program Compilation](#)
 - [Creating, Compiling and Running Your Program](#)
 - [Creating the program](#)
 - [Compilation](#)
 - [Running the program](#)
 - [The C Compilation Model](#)
 - [The Preprocessor](#)
 - [C Compiler](#)
 - [Assembler](#)
 - [Link Editor](#)
 - [Some Useful Compiler Options](#)
 - [Using Libraries](#)
 - [UNIX Library Functions](#)
 - [Finding Information about Library Functions](#)
 - [Lint -- A C program verifier](#)
 - [Exercises](#)
- [C Basics](#)
 - [History of C](#)
 - [Characteristics of C](#)
 - [C Program Structure](#)
 - [Variables](#)
 - [Defining Global Variables](#)
 - [Printing Out and Inputting Variables](#)
 - [Constants](#)
 - [Arithmetic Operations](#)
 - [Comparison Operators](#)
 - [Logical Operators](#)
 - [Order of Precedence](#)
 - [Exercises](#)
- [Conditionals](#)
 - [The if statement](#)
 - [The ? operator](#)
 - [The switch statement](#)
 - [Exercises](#)
- [Looping and Iteration](#)
 - [The for statement](#)
 - [The while statement](#)
 - [The do-while statement](#)
 - [break and continue](#)
 - [Exercises](#)
- [Arrays and Strings](#)
 - [Single and Multi-dimensional Arrays](#)
 - [Strings](#)
 - [Exercises](#)
- [Functions](#)
 - [void functions](#)
 - [Functions and Arrays](#)
 - [Function Prototyping](#)
 - [Exercises](#)
- [Further Data Types](#)
 - [Structures](#)
 - [Defining New Data Types](#)
 - [Unions](#)
 - [Coercion or Type-Casting](#)
 - [Enumerated Types](#)
 - [Static Variables](#)
 - [Exercises](#)
- [Pointers](#)

- [What is a Pointer?](#)
- [Pointer and Functions](#)
- [Pointers and Arrays](#)
- [Arrays of Pointers](#)
- [Multidimensional arrays and pointers](#)
- [Static Initialisation of Pointer Arrays](#)
- [Pointers and Structures](#)
- [Common Pointer Pitfalls](#)
 - [Not assigning a pointer to memory address before using it](#)
 - [Illegal indirection](#)
- [Exercise](#)
- [Dynamic Memory Allocation and Dynamic Structures](#)
 - [Malloc, Sizeof, and Free](#)
 - [Calloc and Realloc](#)
 - [Linked Lists](#)
 - [Full Program: queue.c](#)
 - [Exercises](#)
- [Advanced Pointer Topics](#)
 - [Pointers to Pointers](#)
 - [Command line input](#)
 - [Pointers to a Function](#)
 - [Exercises](#)
- [Low Level Operators and Bit Fields](#)
 - [Bitwise Operators](#)
 - [Bit Fields](#)
 - [Bit Fields: Practical Example](#)
 - [A note of caution: Portability](#)
 - [Exercises](#)
- [The C Preprocessor](#)
 - [#define](#)
 - [#undef](#)
 - [#include](#)
 - [#if -- Conditional inclusion](#)
 - [Preprocessor Compiler Control](#)
 - [Other Preprocessor Commands](#)
 - [Exercises](#)
- [C, UNIX and Standard Libraries](#)
 - [Advantages of using UNIX with C](#)
 - [Using UNIX System Calls and Library Functions](#)
- [Integer Functions, Random Number, String Conversion, Searching and Sorting: <stdlib.h>](#)
 - [Arithmetic Functions](#)
 - [Random Numbers](#)
 - [String Conversion](#)
 - [Searching and Sorting](#)
 - [Exercises](#)
- [Mathematics: <math.h>](#)
 - [Math Functions](#)
 - [Math Constants](#)
- [Input and Output \(I/O\): <stdio.h>](#)
 - [Reporting Errors](#)
 - [perror\(\)](#)
 - [errno](#)
 - [exit\(\)](#)
 - [Streams](#)
 - [Predefined Streams](#)
 - [Redirection](#)
 - [Basic I/O](#)

- [Formatted I/O](#)
 - [Printf](#)
 - [scanf](#)
 - [Files](#)
 - [Reading and writing files](#)
 - [sprintf and sscanf](#)
 - [Stream Status Enquiries](#)
 - [Low Level I/O](#)
 - [Exercises](#)
- [String Handling: <string.h>](#)
 - [Basic String Handling Functions](#)
 - [String Searching](#)
 - [Character conversions and testing: ctype.h](#)
 - [Memory Operations: <memory.h>](#)
 - [Exercises](#)
- [File Access and Directory System Calls](#)
 - [Directory handling functions: <unistd.h>](#)
 - [Scanning and Sorting Directories: <sys/types.h>, <sys/dir.h>](#)
 - [File Manipulation Routines: unistd.h, sys/types.h, sys/stat.h](#)
 - [File Access](#)
 - [errno](#)
 - [File Status](#)
 - [File Manipulation: stdio.h, unistd.h](#)
 - [Creating Temporary Files: <stdio.h>](#)
 - [Exercises](#)
- [Time Functions](#)
 - [Basic time functions](#)
 - [Example time applications](#)
 - [Example 1: Time \(in seconds\) to perform some computation](#)
 - [Example 2: Set a random number seed](#)
 - [Exercises](#)
- [Process Control: <stdlib.h>, <unistd.h>](#)
 - [Running UNIX Commands from C](#)
 - [execl\(\)](#)
 - [fork\(\)](#)
 - [wait\(\)](#)
 - [exit\(\)](#)
 - [Exercises](#)
- [Interprocess Communication \(IPC\), Pipes](#)
 - [Piping in a C program: <stdio.h>](#)
 - [popen\(\) -- Formatted Piping](#)
 - [pipe\(\) -- Low level Piping](#)
 - [Exercises](#)
- [IPC: Interrupts and Signals: <signal.h>](#)
 - [Sending Signals -- kill\(\), _raise\(\)](#)
 - [Signal Handling -- signal\(\)](#)
 - [sig_talk.c -- complete example program](#)
 - [Other signal functions](#)
- [IPC: Message Queues: <sys/msg.h>](#)
 - [Initialising the Message Queue](#)
 - [IPC Functions, Key Arguments, and Creation Flags: <sys/ipc.h>](#)
 - [Controlling message queues](#)
 - [Sending and Receiving Messages](#)
 - [POSIX Messages: <mqueue.h>](#)
 - [Example: Sending messages between two processes](#)
 - [message_send.c -- creating and sending to a simple message queue](#)
 - [message_rec.c -- receiving the above message](#)
 - [Some further example message queue programs](#)

- [msgget.c: Simple Program to illustrate msgget\(\).](#)
 - [msgctl.c: Sample Program to Illustrate msgctl\(\).](#)
 - [msgop.c: Sample Program to Illustrate msgsnd\(\) and msgrcv\(\).](#)
 - [Exercises](#)
- [IPC:Semaphores](#)
 - [Initializing a Semaphore Set](#)
 - [Controlling Semaphores](#)
 - [Semaphore Operations](#)
 - [POSIX Semaphores: <semaphore.h>](#)
 - [semaphore.c: Illustration of simple semaphore passing](#)
 - [Some further example semaphore programs](#)
 - [semget.c: Illustrate the semget\(\) function](#)
 - [semctl.c: Illustrate the semctl\(\) function](#)
 - [semop\(\). Sample Program to Illustrate semop\(\).](#)
 - [Exercises](#)
- [IPC:Shared Memory](#)
 - [Accessing a Shared Memory Segment](#)
 - [Controlling a Shared Memory Segment](#)
 - [Attaching and Detaching a Shared Memory Segment](#)
 - [Example two processes communicating via shared memory: shm_server.c, shm_client.c](#)
 - [shm_server.c](#)
 - [shm_client.c](#)
 - [POSIX Shared Memory](#)
 - [Mapped memory](#)
 - [Address Spaces and Mapping](#)
 - [Coherence](#)
 - [Creating and Using Mappings](#)
 - [Other Memory Control Functions](#)
 - [Some further example shared memory programs](#)
 - [shmget.c: Sample Program to Illustrate shmget\(\).](#)
 - [shmctl.c: Sample Program to Illustrate shmctl\(\).](#)
 - [shmop.c: Sample Program to Illustrate shmat\(\) and shmdt\(\).](#)
 - [Exercises](#)
- [IPC:Sockets](#)
 - [Socket Creation and Naming](#)
 - [Connecting Stream Sockets](#)
 - [Stream Data Transfer and Closing](#)
 - [Datagram sockets](#)
 - [Socket Options](#)
 - [Example Socket Programs: socket_server.c, socket_client](#)
 - [socket_server.c](#)
 - [socket_client.c](#)
 - [Exercises](#)
- [Threads: Basic Theory and Libraries](#)
 - [Processes and Threads](#)
 - [Benefits of Threads vs Processes](#)
 - [Multithreading vs. Single threading](#)
 - [Some Example applications of threads](#)
 - [Thread Levels](#)
 - [User-Level Threads \(ULT\)](#)
 - [Kernel-Level Threads \(KLT\)](#)
 - [Combined ULT/KLT Approaches](#)
 - [Threads libraries](#)
 - [The POSIX Threads Library: libpthread, <pthread.h>](#)
 - [Creating a \(Default\) Thread](#)
 - [Wait for Thread Termination](#)
 - [A Simple Threads Example](#)

- [Detaching a Thread](#)
 - [Create a Key for Thread-Specific Data](#)
 - [Delete the Thread-Specific Data Key](#)
 - [Set the Thread-Specific Data Key](#)
 - [Get the Thread-Specific Data Key](#)
 - [Global and Private Thread-Specific Data Example](#)
 - [Getting the Thread Identifiers](#)
 - [Comparing Thread IDs](#)
 - [Initializing Threads](#)
 - [Yield Thread Execution](#)
 - [Set the Thread Priority](#)
 - [Get the Thread Priority](#)
 - [Send a Signal to a Thread](#)
 - [Access the Signal Mask of the Calling Thread](#)
 - [Terminate a Thread](#)
- [Solaris Threads: <thread.h>](#)
 - [Unique Solaris Threads Functions](#)
 - [Suspend Thread Execution](#)
 - [Continue a Suspended Thread](#)
 - [Set Thread Concurrency Level](#)
 - [Readers/Writer Locks](#)
 - [Readers/Writer Lock Example](#)
 - [Similar Solaris Threads Functions](#)
 - [Create a Thread](#)
 - [Get the Thread Identifier](#)
 - [Yield Thread Execution](#)
 - [Signals and Solaris Threads](#)
 - [Terminating a Thread](#)
 - [Creating a Thread-Specific Data Key](#)
 - [Example Use of Thread Specific Data: Rethinking Global Variables](#)
- [Compiling a Multithreaded Application](#)
 - [Preparing for Compilation](#)
 - [Debugging a Multithreaded Program](#)
- [Further Threads Programming: Thread Attributes \(POSIX\)](#)
 - [Attributes](#)
 - [Initializing Thread Attributes](#)
 - [Destroying Thread Attributes](#)
 - [Thread's Detach State](#)
 - [Thread's Set Scope](#)
 - [Thread Scheduling Policy](#)
 - [Thread Inherited Scheduling Policy](#)
 - [Set Scheduling Parameters](#)
 - [Thread Stack Size](#)
 - [Building Your Own Thread Stack](#)
- [Further Threads Programming: Synchronization](#)
 - [Mutual Exclusion Locks](#)
 - [Initializing a Mutex Attribute Object](#)
 - [Destroying a Mutex Attribute Object](#)
 - [The Scope of a Mutex](#)
 - [Initializing a Mutex](#)
 - [Locking a Mutex](#)
 - [Lock with a Nonblocking Mutex](#)
 - [Destroying a Mutex](#)
 - [Mutex Lock Code Examples](#)
 - [Mutex Lock Example](#)
 - [Using Locking Hierarchies: Avoiding Deadlock](#)
 - [Nested Locking with a Singly Linked List](#)

- [Solaris Mutex Locks](#)
- [Condition Variable Attributes](#)
 - [Initializing a Condition Variable Attribute](#)
 - [Destroying a Condition Variable Attribute](#)
 - [The Scope of a Condition Variable](#)
 - [Initializing a Condition Variable](#)
 - [Block on a Condition Variable](#)
 - [Destroying a Condition Variable State](#)
 - [Solaris Condition Variables](#)
- [Threads and Semaphores](#)
 - [POSIX Semaphores](#)
 - [Basic Solaris Semaphore Functions](#)
- [Thread programming examples](#)
 - [Using `thr\_create\(\)` and `thr\_join\(\)`](#)
 - [Arrays](#)
 - [Deadlock](#)
 - [Signal Handler](#)
 - [Interprocess Synchronization](#)
 - [The Producer / Consumer Problem](#)
 - [A Socket Server](#)
 - [Using Many Threads](#)
 - [Real-time Thread Example](#)
 - [POSIX Cancellation](#)
 - [Software Race Condition](#)
 - [Tgrep: Threaded version of UNIX `grep`](#)
 - [Multithreaded Quicksort](#)
- [Remote Procedure Calls \(RPC\)](#)
 - [What Is RPC](#)
 - [How RPC Works](#)
 - [RPC Application Development](#)
 - [Defining the Protocol](#)
 - [Defining Client and Server Application Code](#)
 - [Compiling and running the application](#)
 - [Overview of Interface Routines](#)
 - [Simplified Level Routine Function](#)
 - [Top Level Routines](#)
 - [Intermediate Level Routines](#)
 - [Expert Level Routines](#)
 - [Bottom Level Routines](#)
 - [The Programmer's Interface to RPC](#)
 - [Simplified Interface](#)
 - [Passing Arbitrary Data Types](#)
 - [Developing High Level RPC Applications](#)
 - [Defining the protocol](#)
 - [Sharing the data](#)
 - [The Server Side](#)
 - [The Client Side](#)
 - [Exercise](#)
- [Protocol Compiling and Lower Level RPC Programming](#)
 - [What is `rpcgen`](#)
 - [An `rpcgen` Tutorial](#)
 - [Converting Local Procedures to Remote Procedures](#)
 - [Passing Complex Data Structures](#)
 - [Preprocessing Directives](#)
 - [`cpreproc` Directives](#)
 - [Compile-Time Flags](#)
 - [Client and Server Templates](#)
 - [Example `rpcgen` compile options/templates](#)

- [Recommended Reading](#)
- [Exercises](#)
- [Writing Larger Programs](#)
 - [Header files](#)
 - [External variables and functions](#)
 - [Scope of externals](#)
 - [Advantages of Using Several Files](#)
 - [How to Divide a Program between Several Files](#)
 - [Organisation of Data in each File](#)
 - [The Make Utility](#)
 - [Make Programming](#)
 - [Creating a makefile](#)
 - [Make macros](#)
 - [Running Make](#)
- [Program Listings](#)
 - [hello.c](#)
 - [printf.c](#)
 - [swap.c](#)
 - [args.c](#)
 - [arg.c](#)
 - [average.c](#)
 - [cio.c](#)
 - [factorial](#)
 - [power.c](#)
 - [ptr_arr.c](#)
 - [Modular Example](#)
 - [main.c](#)
 - [WriteMyString.c](#)
 - [header.h](#)
 - [Makefile](#)
 - [static.c](#)
 - [malloc.c](#)
 - [queue.c](#)
 - [bitcount.c](#)
 - [lowio.c](#)
 - [print.c](#)
 - [cdirc.c](#)
 - [list.c](#)
 - [list_c.c](#)
 - [fork_eg.c](#)
 - [fork.c](#)
 - [signal.c](#)
 - [sig_talk.c](#)
 - [Piping](#)
 - [plot.c](#)
 - [plotter.c](#)
 - [externals.h](#)
 - [random.c](#)
 - [time.c](#)
 - [timer.c](#)

Online Marking of C Programs --- CEILIDH

- [Ceilidh - On Line C Tutoring System](#)
 - [Why Use CEILIDH ?](#)
 - [Introduction](#)

- [Using Ceilidh as a Student](#)
 - [The course and unit level](#)
 - [The exercise level](#)
 - [Interpreted language exercises](#)
 - [Question/answer exercises](#)
 - [The command line interface \(TEXT CEILIDH ONLY\)](#)
 - [Advantages of the command line interface](#)
 - [General points](#)
 - [Conclusions](#)
 - [How Ceilidh works, Ceilidh Course Notes, User Guides etc.](#)
 - [References](#)
- [About this document ...](#)

Dave Marshall
29/3/1999